

Análise Sintática

Eliminação de Recursão à Esquerda

Compiladores — Livro Base: Aho, Lam, Sethi, Ullman (ALSU) (Livro do Dragão)

Conceito de Recursão à Esquerda

Uma gramática é classificada como recursiva à esquerda se ela possui um símbolo não-terminal A capaz de derivar uma forma sentencial que inicia com A .

Definição Formal

Existe uma cadeia de passos de transformação tal que:

$$A \xRightarrow{+} A\alpha \quad \text{para algum } \alpha$$

O Perigo nos Parsers Top-Down

As produções com recursão à esquerda imediata podem causar loops infinitos em analisadores sintáticos descendentes, pois o algoritmo expande continuamente a mesma variável sem consumir tokens.

Eliminação de Recursão à Esquerda Imediata

Regra Geral de Eliminação Imediata

$$\frac{\text{Gramática Original}}{A \rightarrow A\alpha \mid \beta} \quad \Longrightarrow \quad \text{Gramática Livre de Rec. Esq.}$$

Eliminação de Recursão à Esquerda Imediata

Regra Geral de Eliminação Imediata

Gramática Original		Gramática Livre de Rec. Esq.
$A \rightarrow A\alpha \mid \beta$	$\implies$	$A \rightarrow \beta A'$ $A' \rightarrow \alpha A' \mid \varepsilon$

Eliminação de Recursão à Esquerda Imediata

Regra Geral de Eliminação Imediata

$$\frac{\text{Gramática Original}}{A \rightarrow A\alpha \mid \beta} \implies \frac{\text{Gramática Livre de Rec. Esq.}}{A \rightarrow \beta A' \\ A' \rightarrow \alpha A' \mid \varepsilon}$$

Em geral temos

$$\frac{\text{Gramática Original}}{A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \cdots \mid A\alpha_m \mid \beta_1 \mid \beta_2 \mid \cdots \mid \beta_n} \implies \frac{\text{Gramática Livre de Rec. Esq.}}{A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \cdots \mid \beta_n A' \\ A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \cdots \mid \alpha_m A' \mid \varepsilon}$$

Exemplo Prático: Expressões Aritméticas

Aplicação do algoritmo de eliminação de recursão à esquerda imediata na gramática padrão de expressões matemáticas:

Gramática Original Recursiva

$$E \rightarrow E + T \mid T \qquad T \rightarrow T * F \mid F \qquad F \rightarrow (E) \mid \text{ident}$$

Exemplo Prático: Expressões Aritméticas

Aplicação do algoritmo de eliminação de recursão à esquerda imediata na gramática padrão de expressões matemáticas:

Gramática Original Recursiva

$$E \rightarrow E + T \mid T \qquad T \rightarrow T * F \mid F \qquad F \rightarrow (E) \mid \text{ident}$$

Passo a Passo da Eliminação

- Substituição em E ($\alpha = +T$ e $\beta = T$):
 $E \rightarrow TE' \qquad E' \rightarrow +TE' \mid \varepsilon$
- Substituição em T ($\alpha = *F$ e $\beta = F$):
 $T \rightarrow FT' \qquad T' \rightarrow *FT' \mid \varepsilon$

Gramática Final Resultante

$$E \rightarrow TE' \qquad E' \rightarrow +TE' \mid \varepsilon \qquad T \rightarrow FT' \qquad T' \rightarrow *FT' \mid \varepsilon \qquad F \rightarrow (E) \mid \text{ident}$$

Remoção de Produções Vazias (ε -produções)

Regra Geral:

- Se $A \rightarrow \varepsilon$ é uma produção, para cada regra onde A (ou qualquer símbolo anulável) aparece do lado direito, geramos novas produções considerando todas as possibilidades de **manter** ou **remover** cada ocorrência.

Exemplo:

Gramática Original:

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Ac \mid Sd \mid \varepsilon$$

Como $A \rightarrow \varepsilon$, a variável A é anulável.

Remoção de Produções Vazias (ε -produções)

Regra Geral:

- Se $A \rightarrow \varepsilon$ é uma produção, para cada regra onde A (ou qualquer símbolo anulável) aparece do lado direito, geramos novas produções considerando todas as possibilidades de **manter** ou **remover** cada ocorrência.

Exemplo:

Gramática Original:

$$\begin{aligned} S &\rightarrow Aa \mid b \\ A &\rightarrow Ac \mid Sd \mid \varepsilon \end{aligned}$$

Gramática sem ε -produções:

$$\begin{aligned} S &\rightarrow Aa \mid b \mid a \\ A &\rightarrow Ac \mid Sd \mid c \end{aligned}$$

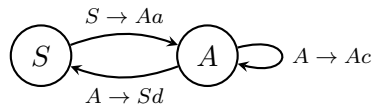
Como $A \rightarrow \varepsilon$, a variável A é anulável.

- Em $S \rightarrow Aa$: mantendo A temos Aa ; omitindo A temos a .
- Em $A \rightarrow Ac$: mantendo A temos Ac ; omitindo A temos c .

Detecção de Recursão à Esquerda via Grafo

Em gramáticas sem ε -produções, constrói-se um **grafo à esquerda**:

- ▶ **Vértices:** Um vértice para cada variável da gramática.
- ▶ **Arcos:** Existe arco $A \rightarrow B$ no grafo se houver produção $A \rightarrow B\alpha$ na gramática.



Condição de Recursão à Esquerda

Haverá recursão à esquerda na gramática se e somente se houver um **ciclo dirigido** no grafo.

- ▶ **Ciclo no Grafo:** $S \rightarrow A \rightarrow S$
- ▶ **Sequência de Derivação:**

$$S \Rightarrow Aa \Rightarrow Sda$$

(comprova a recursão à esquerda indireta)

Detecção de Recursão à Esquerda via Grafo

Lema: Existe recursão à esquerda em uma gramática se e somente se existe um ciclo dirigido no grafo correspondente.

Demonstração

($\Rightarrow$) Se existe recursão à esquerda na gramática, então o grafo possui um ciclo dirigido.

Detecção de Recursão à Esquerda via Grafo

Lema: Existe recursão à esquerda em uma gramática se e somente se existe um ciclo dirigido no grafo correspondente.

Demonstração

($\Rightarrow$) Se existe recursão à esquerda na gramática, então o grafo possui um ciclo dirigido. Como a gramática possui recursão à esquerda, então existe uma sequência de derivações da seguinte forma

$A_0 \Rightarrow A_1\alpha_1 \Rightarrow A_2\alpha_2\alpha_1 \Rightarrow \dots A_{n-1}\alpha_{n-1} \dots \alpha_1 \Rightarrow A_0\alpha_n \dots \alpha_1$. Logo, existem produções na gramática $A_0 \rightarrow A_1\alpha_1, A_1 \rightarrow A_2\alpha_2, \dots, A_{n-1} \rightarrow A_0\alpha_n$. Com isso, pela construção do grafo, existe o seguinte ciclo dirigido $A_0 \rightarrow A_1 \rightarrow A_2 \dots \rightarrow A_{n-1} \rightarrow A_0$.

Detecção de Recursão à Esquerda via Grafo

Lema: Existe recursão à esquerda em uma gramática se e somente se existe um ciclo dirigido no grafo correspondente.

Demonstração

($\Leftarrow$) Se existe um ciclo dirigido no grafo de uma gramática, então essa gramática é recursiva à esquerda.

Detecção de Recursão à Esquerda via Grafo

Lema: Existe recursão à esquerda em uma gramática se e somente se existe um ciclo dirigido no grafo correspondente.

Demonstração

($\Leftarrow$) Se existe um ciclo dirigido no grafo de uma gramática, então essa gramática é recursiva à esquerda.

Considere o ciclo $A_0 \rightarrow A_1 \rightarrow A_2 \cdots \rightarrow A_{n-1} \rightarrow A_0$ no grafo de uma gramática. Então existem produções na gramática da seguinte forma

$A_0 \rightarrow A_1\alpha_1, A_1 \rightarrow A_2\alpha_2, \dots, A_{n-1} \rightarrow A_0\alpha_n$. Com isso, existe uma sequência de derivação que mostra que a gramática é recursiva à esquerda:

$A_0 \Rightarrow A_1\alpha_1 \Rightarrow A_2\alpha_2\alpha_1 \Rightarrow \dots A_{n-1}\alpha_{n-1} \dots \alpha_1 \Rightarrow A_0\alpha_n \dots \alpha_1$.

□

Algoritmo Geral de Eliminação de Recursão à Esquerda

O algoritmo geral resolve casos complexos onde a recursão à esquerda ocorre de forma indireta através de ciclos entre múltiplos não-terminais.

Pré-requisitos de Entrada

A gramática G de entrada não deve possuir ciclos da forma $A \xRightarrow{+} A$ e deve ser livre de produções vazias ($A \rightarrow \varepsilon$).

Estrutura do Algoritmo

1. Ordene os não-terminais em uma sequência arbitrária $A_1, A_2, \dots, A_n$.
2. **para** $i = 1$ **até** n **faça**
3. **para** $j = 1$ **até** $i - 1$ **faça**
4. Substitua cada produção da forma $A_i \rightarrow A_j \gamma$ pelas produções de A_j .
5. Elimine as recursões à esquerda imediatas existentes nas produções de A_i .

Corretude do algoritmo

- ▶ A linha 1 coloca os vértices do grafo em uma ordem específica.

Corretude do algoritmo

- ▶ A linha 1 coloca os vértices do grafo em uma ordem específica.
- ▶ **Invariante:** No início da linha 2, vale que um arco que sai de A_k com $k = 1, 2, \dots, i - 1$, entra em A_l com $l > k$.

Corretude do algoritmo

- ▶ A linha 1 coloca os vértices do grafo em uma ordem específica.
- ▶ **Invariante:** No início da linha 2, vale que um arco que sai de A_k com $k = 1, 2, \dots, i - 1$, entra em A_l com $l > k$.
- ▶ Se existe ciclo dirigido no grafo, então existe um arco $A_i \rightarrow A_j$ “que volta”.

Corretude do algoritmo

- ▶ A linha 1 coloca os vértices do grafo em uma ordem específica.
- ▶ **Invariante:** No início da linha 2, vale que um arco que sai de A_k com $k = 1, 2, \dots, i - 1$, entra em A_l com $l > k$.
- ▶ Se existe ciclo dirigido no grafo, então existe um arco $A_i \rightarrow A_j$ “que volta”.
 - ▶ Como A_j é substituído por suas produções (linha 4) e vale o invariante, então o arco com ponta inicial em A_i que caracteriza o ciclo é “movido para frente” (ou eliminado).

Corretude do algoritmo

- ▶ A linha 1 coloca os vértices do grafo em uma ordem específica.
- ▶ **Invariante:** No início da linha 2, vale que um arco que sai de A_k com $k = 1, 2, \dots, i - 1$, entra em A_l com $l > k$.
- ▶ Se existe ciclo dirigido no grafo, então existe um arco $A_i \rightarrow A_j$ “que volta”.
 - ▶ Como A_j é substituído por suas produções (linha 4) e vale o invariante, então o arco com ponta inicial em A_i que caracteriza o ciclo é “movido para frente” (ou eliminado).
- ▶ O processo de “mover o arco para frente” pode causar uma recursão à esquerda imediata em A_i , e isso é tratado na linha 5.

Corretude do algoritmo

- ▶ A linha 1 coloca os vértices do grafo em uma ordem específica.
- ▶ **Invariante:** No início da linha 2, vale que um arco que sai de A_k com $k = 1, 2, \dots, i - 1$, entra em A_l com $l > k$.
- ▶ Se existe ciclo dirigido no grafo, então existe um arco $A_i \rightarrow A_j$ “que volta”.
 - ▶ Como A_j é substituído por suas produções (linha 4) e vale o invariante, então o arco com ponta inicial em A_i que caracteriza o ciclo é “movido para frente” (ou eliminado).
- ▶ O processo de “mover o arco para frente” pode causar uma recursão à esquerda imediata em A_i , e isso é tratado na linha 5.
- ▶ No fim do algoritmo, o grafo não terá “arcos que voltam”. Logo, o grafo não terá ciclos dirigidos e a gramática correspondente não possuirá recursão à esquerda.

Execução do Algoritmo Geral: Caso Prático

Aplicação do algoritmo geral em uma gramática com recursão indireta.

Gramática de Entrada G'

$$S \rightarrow Aa \mid b \mid a$$

$$A \rightarrow Ac \mid Sd \mid c$$

Ordenação dos não-terminais: (S, A) .

Iteração $i = 1$ (S)

O laço interno não executa. Nenhuma recursão imediata é encontrada em S .

Iteração $i = 2$ (A)

Roda para $j = 1$. Substitui-se $A \rightarrow Sd$ pelas regras de S :

$$A \rightarrow Ac \mid Aad \mid bd \mid ad \mid c$$

Agora existe recursão à esquerda imediata.

Execução do Algoritmo Geral: Caso Prático

Eliminação da Recursão Imediata em A

Isolando os fatores recursivos ($\alpha_1 = c$, $\alpha_2 = ad$) com a variável auxiliar A' :

$$S \rightarrow Aa \mid b \mid a \quad A \rightarrow bdA' \mid adA' \mid cA' \quad A' \rightarrow cA' \mid adA' \mid \varepsilon$$